

Anexo D: Integración del Sistema, Software de Adquisición de Datos y Control

Santiago Rosas Mogollón

Marzo 2025

1. Descripción General de la Integración Hardware-Software

Este documento complementario detalla la arquitectura de firmware y control en Python utilizada para la sincronización del subsistema óptico y mecánico. El objetivo principal es desglosar de forma transparente el funcionamiento del firmware analógico de la celda de carga/sensor de tensión, el generador de pulsos del motor a través del microcontrolador Arduino, y el script maestro de control basado en Python, preservando las estructuras de flujo originales analizadas durante el proceso de desarrollo y depuración (*debugging*).

2. Software de Adquisición de Datos de Tensión

El primer módulo de control de bajo nivel se implementó sobre una plataforma Arduino con la finalidad de leer los cambios transductores analógicos proporcionales a la tensión mecánica ejercida en el sistema.

2.1. Procesamiento Digital de Señal y Ecuación de Conversión

El convertidor analógico-digital (ADC) de la placa opera con una resolución nativa de 10 bits, lo que mapea los voltajes de entrada entre 0 V y 5 V a un rango discreto de enteros $[0, 1023]$.

La conversión e instrumentación matemática implementada en el firmware sigue la siguiente ecuación explícita:

$$\text{Fuerza} = \text{valorLeido} \times \left(\frac{5,0}{1023,0} \right) \times 97,54 \quad (1)$$

Donde la constante 97,54 actúa como el factor de escala o ganancia de calibración lineal del sistema para transformar el voltaje medido directamente a unidades de fuerza o tensión. El retardo del bucle se fija en $\Delta t = 300$ ms mediante la función `delay(300)`, definiendo una tasa de muestreo aproximada de 3,33 Hz.

3. Software de Control del Motor

El segundo microcontrolador está dedicado de manera exclusiva a la modulación y generación de los pulsos digitales requeridos por el Driver Rtelligent para comandar el motor.

3.1. Modulación de Frecuencia por Software y Entrada Serial

Al arrancar el módulo, se parametriza una frecuencia nominal base de 200 Hz, traducándose a microsegundos mediante la relación:

$$\text{periodo} = \frac{1,000,000}{\text{frecuencia}} \quad (2)$$

El bucle de control monitorea continuamente el buffer serial a 115200 bps. Al recibir una cadena de texto a través del bus, el programa discrimina entre dos rutas lógicas secuenciales:

- Si la cadena es exactamente igual a la palabra clave "STOP", conmuta el pin digital a nivel bajo (LOW) para detener el movimiento.
- De lo contrario, procesa la cadena transformándola a un tipo numérico entero (`input.toInt()`) y actualiza en tiempo real el valor del período de la onda cuadrada.

4. Software de Control Centralizado en Python

El programa maestro implementado en Python actúa como el núcleo de sincronización. Este integra la visión artificial por computadora (vía OpenCV), la adquisición de fuerza a través de puertos serie y la generación automática de la memoria de datos en formato Excel.

4.1. Configuración de ORB

En esta etapa de integración el compute de ORB se parametriza explícitamente con los siguientes coeficientes operativos para afinar la detección de keypoints en la Región de Interés (ROI):

- `nfeatures=500`: Limita la extracción a un máximo de 500 puntos clave para resguardar la latencia de procesamiento.
- `scaleFactor=1.2` y `nlevels=8`: Estructura una pirámide de escala de 8 niveles con un factor de reducción del 20 % por nivel para otorgar tolerancia multiescala.
- `scoreType=cv2.ORB_HARRIS_SCORE`: Utiliza el algoritmo clásico de Harris para rankear los puntos de interés en función de la matriz de covarianza de gradientes locales.

4.2. Flujo de Control Multi-Puerto (Serial) y Sincronización

El script realiza una apertura paralela de dos puertos de comunicación independientes sobre el sistema operativo: COM4 (dedicado a la escucha pasiva del flujo de lecturas de tensión de la celda) y COM3 (dedicado al envío activo de comandos de control de velocidad hacia el controlador del motor).

Apéndice I: Código de Adquisición de Datos de Tensión (Arduino)

```
1 // Definir el pin analógico que se usará
2 const int pinAnalogico = A0; // Puedes usar A1, A2, etc.
3 int valorLeido = 0; // Variable para almacenar el valor de lectura
4 float Fuerza = 0.0; // Variable para almacenar el voltaje real
5
6 void setup() {
7   // Inicia la comunicación serial para ver el resultado en el Monitor
   Serial
8   Serial.begin(115200);
9 }
10
11 void loop() {
12   // Lee el valor del pin analógico
13   valorLeido = analogRead(pinAnalogico);
14
15   // Convierte el valor leído a un voltaje real (0 a 5 V)
16   Fuerza = valorLeido * (5.0 / 1023.0)*97.54; // Fórmula de
   conversión
17
18   // Muestra el valor leído y el voltaje calculado en el Monitor
   Serial
19
20   Serial.print("Valor leído: ");
21   Serial.print(valorLeido);
22   Serial.print(" Fuerza: ");
23   Serial.println(Fuerza);
24
25   // Espera un segundo antes de leer nuevamente
26   delay(300);
27 }
```

Apéndice II: Código de Control del Motor (Arduino)

```
1 // Definir el pin donde se generar el pulso
2 const int pinPulso = 9; // Pin 9 es v lido en Arduino Uno
3 // Variable para almacenar la frecuencia deseada (en Hz)
4 int frecuencia = 200; // Frecuencia inicial
5
6 // Variable para almacenar el per odo del pulso (en microsegundos)
7 unsigned long periodo;
8
9 void setup() {
10 // Configurar el pin como salida
11 pinMode(pinPulso, OUTPUT);
12 digitalWrite(pinPulso, LOW);
13 // Iniciar la comunicaci n serial
14 Serial.begin(115200);
15 // Calcular el per odo inicial
16 periodo = 1000000 / frecuencia; // 1 segundo = 1,000,000
17   microsegundos
18 }
19 void loop() {
20   if (Serial.available() > 0) {
21     String input = Serial.readStringUntil('\n');
22
23     if (input == "STOP") {
24       // Detener completamente los pulsos
25       digitalWrite(pinPulso, LOW);
26     } else {
27       // Procesar como frecuencia
28       frecuencia = input.toInt();
29       periodo = 1000000 / frecuencia;
30     }
31   }
32
33   if (frecuencia > 0) { // Solo generar pulsos si frecuencia > 0
34     digitalWrite(pinPulso, HIGH);
35     delayMicroseconds(periodo / 2);
36     digitalWrite(pinPulso, LOW);
37     delayMicroseconds(periodo / 2);
38   } else {digitalWrite(pinPulso, LOW);}
39 }
```

Apéndice III: Código de Control Principal (Python)

```
1 import cv2
2 import numpy as np
3 import time
4 from openpyxl import Workbook
5 from datetime import datetime
6 import serial
7
8 # Configuración de visualización
9 DISPLAY_WIDTH = 640
10 DISPLAY_HEIGHT = 480
11 #resolución
12 RESOLUCION = (1280, 720)
13 CALIDAD = 80
14
15 MOTOR_FREQUENCY = 200 # Frecuencia inicial del motor en Hz
16
17 # Inicializar ORB
18 orb = cv2.ORB_create(nfeatures=500,
19                      scaleFactor=1.2,
20                      nlevels=8,
21                      edgeThreshold=31,
22                      firstLevel=0,
23                      WTA_K=2,
24                      scoreType=cv2.ORB_HARRIS_SCORE,
25                      patchSize=31,
26                      fastThreshold=20)
27
28 # Inicializar la cámara principal
29 cap = cv2.VideoCapture(1) # Usar la cámara predeterminada (cámara web)
30
31 # Inicializar la segunda cámara
32 cap2 = cv2.VideoCapture(0) # Usar la segunda cámara
33
34 # Verificar si las cámaras se abrieron correctamente
35 if not cap.isOpened():
36     print("Error: No se pudo abrir la cámara principal.")
37     exit()
38
39 if not cap2.isOpened():
40     print("Error: No se pudo abrir la segunda cámara.")
41     exit()
42
43 #Configurador de resolución a 1080p para ambas cámaras
44 cap.set(cv2.CAP_PROP_FRAME_WIDTH, RESOLUCION[0])
45 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, RESOLUCION[1])
46
47 cap2.set(cv2.CAP_PROP_FRAME_WIDTH, RESOLUCION[0])
48 cap2.set(cv2.CAP_PROP_FRAME_HEIGHT, RESOLUCION[1])
49
50 # Obtener la resolución del frame de la cámara principal
51 frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
52 frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
53 fps = int(cap.get(cv2.CAP_PROP_FPS))
54
55 # Obtener la resolución del frame de la segunda cámara
```

```

56 frame_width2 = int(cap2.get(cv2.CAP_PROP_FRAME_WIDTH))
57 frame_height2 = int(cap2.get(cv2.CAP_PROP_FRAME_HEIGHT))
58 fps2 = int(cap2.get(cv2.CAP_PROP_FPS))
59
60 # Inicializar VideoWriter para guardar el video de la cámara principal
61 video_output = cv2.VideoWriter('salida_video_macro.mp4', cv2.
    VideoWriter_fourcc(*'mp4v'), fps, RESOLUCION)
62 video_output.set(cv2.VIDEOWRITER_PROP_QUALITY, CALIDAD)
63 # Inicializar VideoWriter para guardar el video de la segunda cámara
64 video_output2 = cv2.VideoWriter('salida_video_micro.mp4', cv2.
    VideoWriter_fourcc(*'mp4v'), fps2, RESOLUCION)
65 video_output2.set(cv2.VIDEOWRITER_PROP_QUALITY, CALIDAD)
66
67 # Variables para almacenar el frame de referencia
68 reference_frame = None
69 reference_keypoints = None
70 reference_descriptors = None
71 roi = None # Variable para almacenar la ROI
72
73 # Variable para controlar el tiempo
74 last_time = time.time()
75 interval = 0.3 # Intervalo de 0.3 segundo
76
77 # Umbral para ignorar desplazamientos pequeños (en píxeles)
78 displacement_threshold = 1 # Ignorar desplazamientos menores a 1 píxel
79
80 # Crear un archivo Excel para guardar los datos
81 wb = Workbook()
82 ws = wb.active
83 ws.title = "Deformación"
84 ws.append(["Timestamp", "Desplazamiento (píxeles)", "Valor leído", "
    Newtons"]) # Encabezados
85
86 # Variable para controlar la pausa
87 paused = False
88 puerto_serial = serial.Serial('COM4', 115200, timeout=1)
89 puerto_serial2 = serial.Serial('COM3', 115200, timeout=1)
90
91 # Función para controlar el motor
92 def control_motor(frequency=None):
93     if frequency is not None:
94         puerto_serial2.write(f"{frequency}\n".encode())
95     else:
96         puerto_serial2.write("STOP\n".encode())
97
98 # Función para mantener el frame de referencia y seleccionar la ROI
99 def capture_reference_frame():
100     global reference_frame, reference_keypoints, reference_descriptors,
        roi
101     ret, frame = cap.read()
102     if not ret:
103         print("Error: No se pudo capturar el frame de referencia.")
104         return False
105
106     # Mostrar el frame y permitir al usuario seleccionar la ROI
107     roi = cv2.selectROI("Selecciona la ROI", frame, fromCenter=False,
        showCrosshair=True)
108     cv2.destroyWindow("Selecciona la ROI")

```

```

109
110     # Recortar el frame de referencia seg n la ROI
111     reference_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
112     reference_frame = reference_frame[int(roi[1]):int(roi[1] + roi[3]),
113                                     int(roi[0]):int(roi[0] + roi[2])]
114
115     # Detectar puntos caracter sticos y descriptores en la ROI
116     reference_keypoints, reference_descriptors = orb.detectAndCompute(
117         reference_frame, None)
118     if reference_descriptors is None:
119         print("Error: No se pudieron calcular los descriptores de
120             referencia.")
121         return False
122     print("Frame de referencia capturado y ROI seleccionada.")
123     return True
124     control_motor(MOTOR_FREQUENCY)
125
126 # Capturar el frame de referencia y seleccionar la ROI inicial
127 if not capture_reference_frame():
128     cap.release()
129     cap2.release()
130     cv2.destroyAllWindows()
131     exit()
132
133 last_time = time.time()
134 interval = 0.3
135 last_valid_displacement = 0.0
136 newtons = 0.0
137 valor_leido = 0.0
138
139 while True:
140     ret, frame = cap.read()
141     if not ret:
142         print("Error: No se pudo capturar el frame de la c mara
143             principal.")
144         break
145     frame_display = cv2.resize(frame, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
146
147     ret2, frame2 = cap2.read()
148     if not ret2:
149         print("Error: No se pudo capturar el frame de la segunda c mara
150             .")
151         break
152     frame2_display = cv2.resize(frame2, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
153
154     frame_with_roi = frame.copy()
155     if roi is not None:
156         cv2.rectangle(frame_with_roi, (int(roi[0]), int(roi[1])), (int(
157             roi[0] + roi[2]), int(roi[1] + roi[3])), (0, 255, 0), 2)
158
159     if paused:
160         cv2.putText(frame_with_roi, "PAUSADO", (10, 50), cv2.
161             FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
162
163     cv2.imshow('Camera meso', frame_with_roi)
164
165     if paused:
166         frame2_paused = frame2.copy()

```

```

160     cv2.putText(frame2_paused, "PAUSADO", (10, 50), cv2.
FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
161     cv2.imshow('Camera micro', frame2_paused)
162     else:
163         cv2.imshow('Camera micro', frame2)
164
165     if not paused:
166         video_output.write(frame)
167         video_output2.write(frame2)
168
169     current_time = time.time()
170
171     if not paused and current_time - last_time >= interval:
172         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
173         gray_roi = gray[int(roi[1]):int(roi[1] + roi[3]), int(roi[0]):
int(roi[0] + roi[2])]
174
175         keypoints, descriptors = orb.detectAndCompute(gray_roi, None)
176
177         if descriptors is not None and reference_descriptors is not None
:
178             bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
179             matches = bf.match(reference_descriptors, descriptors)
180
181             average_displacement = 0.0
182             if len(matches) > 0:
183                 matches = sorted(matches, key=lambda x: x.distance)
184
185                 displacements = []
186                 for match in matches:
187                     pt1 = reference_keypoints[match.queryIdx].pt
188                     pt2 = keypoints[match.trainIdx].pt
189                     dx = pt2[0] - pt1[0]
190                     dy = pt2[1] - pt1[1]
191                     displacement = np.sqrt(dx**2 + dy**2)
192                     displacements.append(displacement)
193
194                 average_displacement = np.mean(displacements)
195
196                 if average_displacement > displacement_threshold:
197                     print(f"Desplazamiento promedio: {
average_displacement:.2f} p xeles")
198                     last_valid_displacement = average_displacement
199                 else:
200                     print("Desplazamiento despreciable (ruido)")
201                     average_displacement = last_valid_displacement
202             else:
203                 print("No se encontraron matches. Usando el ltimo
desplazamiento v lido.")
204                 average_displacement = last_valid_displacement
205
206             if len(matches) > 0:
207                 ref_resized = cv2.resize(reference_frame, (DISPLAY_WIDTH
// 2, DISPLAY_HEIGHT // 2))
208                 current_resized = cv2.resize(gray_roi, (DISPLAY_WIDTH //
2, DISPLAY_HEIGHT // 2))
209                 output_frame = cv2.drawMatches(ref_resized,
reference_keypoints, current_resized, keypoints, matches[:50], None,

```

```

210         flags=2)
211         output_frame = cv2.resize(output_frame, (DISPLAY_WIDTH,
212         DISPLAY_HEIGHT // 2))
213         cv2.imshow('Matches', output_frame)
214     else:
215         print("No se pudieron calcular los descriptores. Usando el
216         ltimo desplazamiento v lido.")
217         average_displacement = last_valid_displacement
218
219     linea = puerto_serial.readline().decode('utf-8').strip()
220     if linea:
221         partes = linea.split()
222         if len(partes) >= 5:
223             valor_leido = float(partes[2])
224             newtons = float(partes[4])
225             print(f"Fuerza:{newtons:.2f}")
226             timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%
227 f")
228             ws.append([timestamp, average_displacement, valor_leido,
229             newtons])
230
231     last_time = current_time
232
233     key = cv2.waitKey(1)
234     if key & 0xFF == ord('p'):
235         control_motor(None)
236         paused = not paused
237         if not paused:
238             if not capture_reference_frame():
239                 break
240             print("Pausado" if paused else "Continuando con nueva referencia
241 ")
242
243     if key & 0xFF == ord('q'):
244         control_motor(None)
245         break
246
247 output_file = " analisis_deformaci n.xlsx"
248 wb.save(output_file)
249 print(f"Datos guardados en {output_file}")
250
251 cap.release()
252 video_output.release()
253 cap2.release()
254 video_output2.write(frame2)
255 video_output2.release()
256 cv2.destroyAllWindows()
257
258 puerto_serial.close()
259 puerto_serial2.close()

```